```
                    break
                with self._status_lock:
                    self._bind_error = str(exc)
                    self._is_connected = False
                print(
                    f"[PingStation] Socket error on port {self.port}: {exc} ? "
                    f"retrying in 3 s ?"
                )
                time.sleep(3.0)
            finally:
                with self._socket_lock:
                    self._server_socket = None
                if server is not None:
                    try:
                        server.close()
                    except OSError:
                        pass

    # ------------------------------------------------------------------
    def _handle_connection(self, conn, addr):
        """Process all bytes from a single connected PingStation client."""
        conn.settimeout(1.0)
        buffer_bytes = b""

        with conn:
            while not self._stop_event.is_set():
                try:
                    chunk = conn.recv(16384)
                except socket.timeout:
                    continue
                except OSError:
                    break

                if not chunk:
                    break

                if self.raw_logger is not None:
                    try:
                        self.raw_logger.log_chunk(chunk, addr[0], addr[1])
                    except OSError:
                        pass

                buffer_bytes += chunk
                frames, buffer_bytes = self.decoder.split_frames(buffer_bytes)
                if not frames:
                    continue

                with self._lock:
                    for frame in frames:
                        record = self.decoder.decode_frame(frame, source_ip=addr[0])
                        if record is not None:
                            self._pending_records.append(record)
                            if self.raw_logger is not None:
                                try:
                                    self.raw_logger.log_decoded(record)
                                except OSError:
                                    pass

                with self._status_lock:
                    self._last_data_time = time.time()
                    self._total_frames_decoded += len(frames)

    # ------------------------------------------------------------------
    def get_snapshot(self):
        with self._lock:
            return self.decoder.get_state_snapshot(stale_seconds=self.stale_seconds)

    def drain_records(self):
        """Return each newly decoded record once for persistent storage."""
        with self._lock:
            rows = self._pending_records
            self._pending_records = []
            return rows
```